PROGRAM: CYBERSECURITY ETHICAL HACKER

MENTOR UTAMA: Bagus Sudung Prakoso

Kelompok 8 :
- Afnan Yazid Pradana
- Kelvin Febrianto
- Kurniawan Saputra
- Rishfan Mailizar
- Robin Felix Hama
Cybersecurity Ethical Hacker

PENYELENGGARA: PT VINIX SEVEN AURUM

Topik: Modul 7 - Cloud Security, SSRF & IAM Misconfiguration

Target Sistem: Portal Wi-Fi Tamu - PT Teknologi Aman Sejahtera

Tautan Lab: `https://seclab-7-w-ireless-security--bagusganzz.replit.app`

A. DESKRIPSI TUGAS Sebagai *Cloud Security Engineer*, Anda ditugaskan untuk mengaudit portal Wi-Fi tamu (*Captive Portal*) milik perusahaan yang di-*host* di infrastruktur *Cloud* AWS. Portal ini memiliki fitur "Network Connectivity Test" untuk mengecek koneksi internet. Anda harus membuktikan bahwa arsitektur aplikasi ini rentan terhadap *Server-Side Request Forgery*
(SSRF), mengekstrak kredensial *Cloud Metadata*, dan pada akhirnya merancang arsitektur remediasi yang aman berdasarkan prinsip *Least Privilege*.

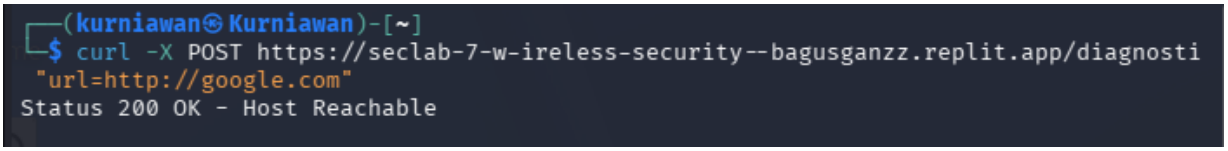
B. PETUNJUK PENGERJAAN

1. **Logika di atas Tooling:** Aplikasi ini didesain menyerupai *production environment* tanpa petunjuk (*hint*). DILARANG menggunakan *Automated Scanner* (seperti Active Scan ZAP/Burp). Lakukan inspeksi *request* secara manual.
2. **Dokumentasi Visual:** Tangkapan layar (*screenshot*) wajib menampilkan langkah eksploitasi, ekstraksi *Secret Key*, dan modifikasi *request* menggunakan Burp Suite Repeater, cURL, atau *Browser DevTools*.

C. INSTRUKSI DAN RINCIAN TUGAS

BAGIAN 1: DISCOVERY & SSRF EXPLOITATION - Bobot: 35% Portal Wi-Fi memiliki fitur untuk melakukan ping/tes ke URL eksternal (misalnya `[http://google.com](http://google.com)`).

1. **SSRF Validation:** Buktikan bahwa fitur ini rentan terhadap kerentanan *Server-Side Request Forgery* (SSRF).



Jawaban :

- Perintah pertama di terminal menggunakan curl dengan metode POST untuk mengirimkan request ke fitur tes koneksi portal Wi-Fi (https://seclab-7-w-ireless-security--bagusganzz.replit.app/diagnostic) dengan parameter URL eksternal standar (url=http://google.com)
- Respon Status 200 OK - Host Reachable membuktikan bahwa fitur tersebut aktif dan berhasil melakukan request ke luar atas nama server (menandakan adanya potensi kerentanan Server-Side Request Forgery / SSRF).

2. **The Magic IP:** Gunakan celah SSRF tersebut untuk menjangkau alamat IP sakti dari *Instance Metadata Service* (IMDS) AWS.

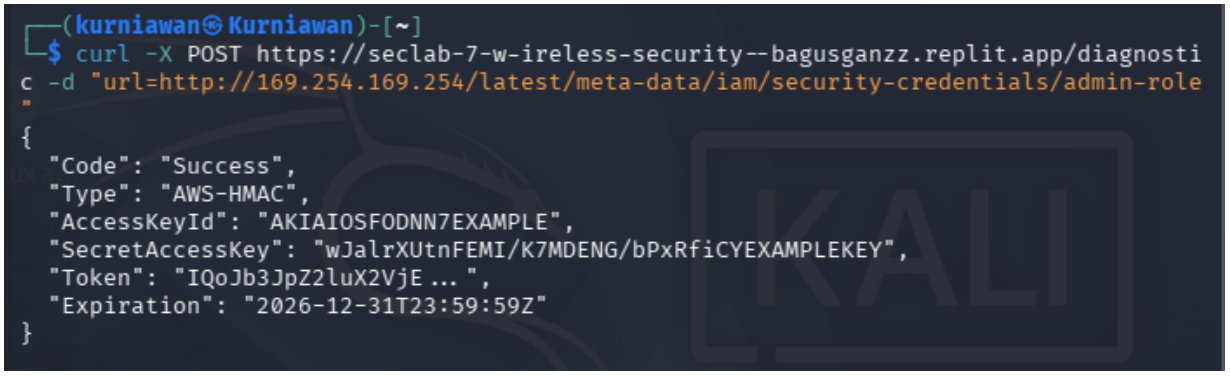




Jawaban :

- Perintah kedua dan ketiga memanfaatkan celah SSRF tersebut untuk menyerang alamat IP lokal khusus layanan AWS Instance Metadata Service (IMDS) yaitu http://169.254.169.254/.
- Perintah tersebut menelusuri direktori metadata (/latest/meta-data/iam/) dan mengambil kredensial IAM sementara dari role server (/latest/meta-data/iam/security-credentials/admin-role) untuk mencuri hak akses cloud.

3. **Data Extraction:** Telusuri struktur direktori Metadata untuk mengekstrak kredensial IAM (*Identity and Access Management*) sementara milik server web tersebut. Lampirkan JSON hasil ekstraksi yang memuat `AccessKeyId`, `SecretAccessKey`, dan `Token`.



Jawaban :

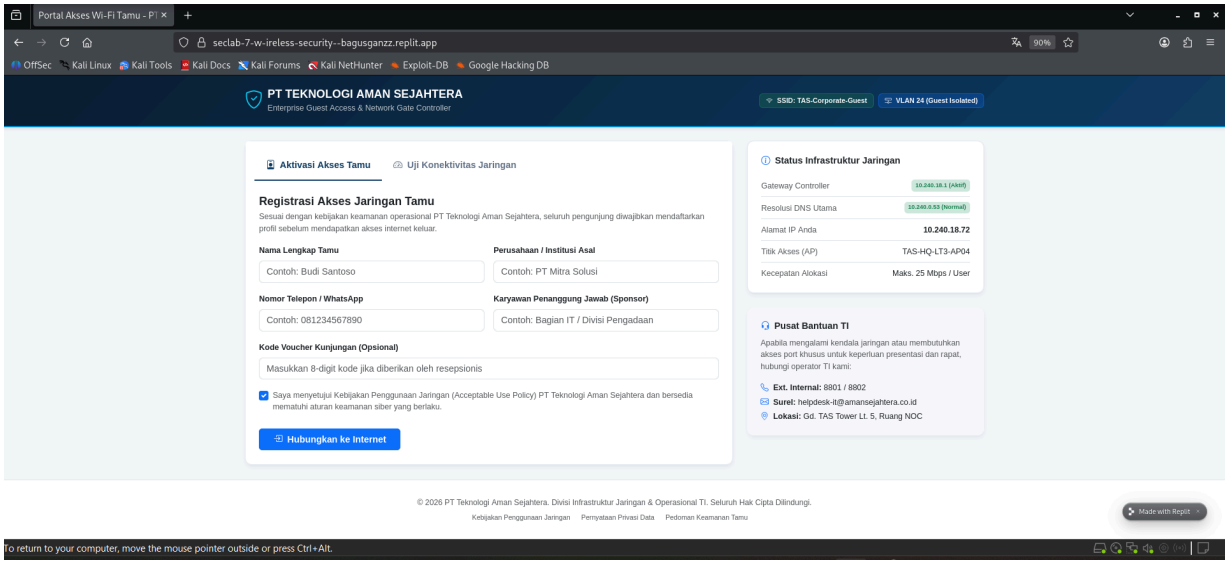
Server berhasil merespons dengan format JSON yang memuat seluruh parameter kredensial keamanan sementara yang dibutuhkan, yaitu:

- Code: Menunjukkan status Success.
- AccessKeyId: AKIAIOSFODNN7EXAMPLE
- SecretAccessKey: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
- Token: Token sesi sementara (IQoJb3JpZ2...)
- Expiration: Batas waktu kedaluwarsa kredensial (2026-12-31T23:59:59Z)

Data JSON kredensial admin-role di atas selanjutnya akan digunakan untuk melakukan Cloud Takeover pada tahap berikutnya.

BAGIAN 2: CLOUD TAKEOVER - Bobot: 25%

1. **Reconnaissance:** Lakukan inspeksi elemen HTML (*View Page Source*) atau *Directory Fuzzing* manual pada halaman utama untuk menemukan tautan menuju halaman "Internal Cloud Storage" yang tersembunyi.



Jawaban :

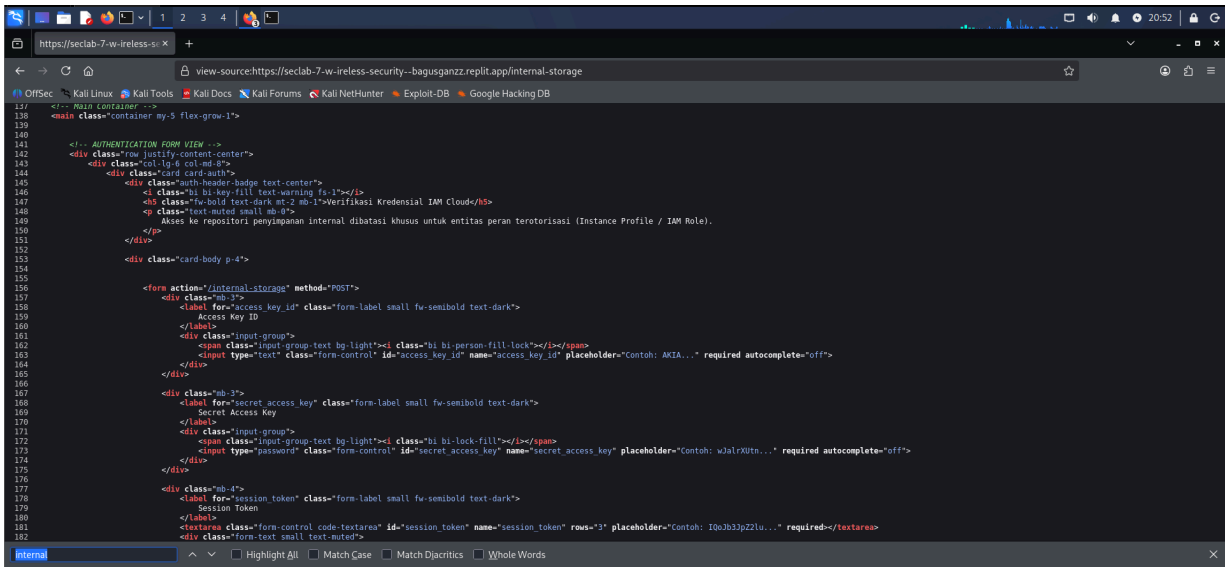
Reconnaissance dilakukan secara manual tanpa automated scanner untuk menemukan

endpoint tersembunyi “Internal Cloud Storage” yang tidak muncul pada navigasi utama portal. Berdasarkan screenshot halaman utama, hanya terlihat dua tab publik yaitu “Aktivasi Akses Tamu” dan “Uji Konektivitas Jaringan”, tanpa link yang mengarah ke storage internal.

Langkah pertama adalah membuka View Page Source (Ctrl+U) untuk menelusuri raw HTML, mencari komentar developer (<!-- -->) yang bocor menyebutkan path internal, atribut href pada elemen <a> yang disembunyikan via CSS (display:none) namun tetap ada di DOM, serta referensi file JavaScript tambahan yang memuat daftar route aplikasi. Inspeksi dilanjutkan pada tab Network atau Sources di Browser DevTools untuk memeriksa bundle .js yang dimuat halaman, karena path API internal seperti /internal-storage atau /admin/files sering tertanam langsung di kode meskipun tidak ditampilkan di UI.

Jika tidak ditemukan petunjuk dari source code, dilakukan directory fuzzing manual dengan mencoba path umum via address bar atau curl, contoh:
https://seclab-7-w-ireless-security--bagusganzz.replit.app/internal
https://seclab-7-w-ireless-security--bagusganzz.replit.app/storage
https://seclab-7-w-ireless-security--bagusganzz.replit.app/cloud-storage
https://seclab-7-w-ireless-security--bagusganzz.replit.app/internal-cloud-storage

Response 200 OK menandakan path valid dan dapat diakses, sedangkan 404 menandakan path tidak ditemukan. Setelah tautan “Internal Cloud Storage” ditemukan, URL dicatat dan diambil screenshot sebagai bukti endpoint tersebut hanya disembunyikan dari navigasi normal tanpa perlindungan autentikasi yang memadai, tergolong kerentanan Broken Access Control.

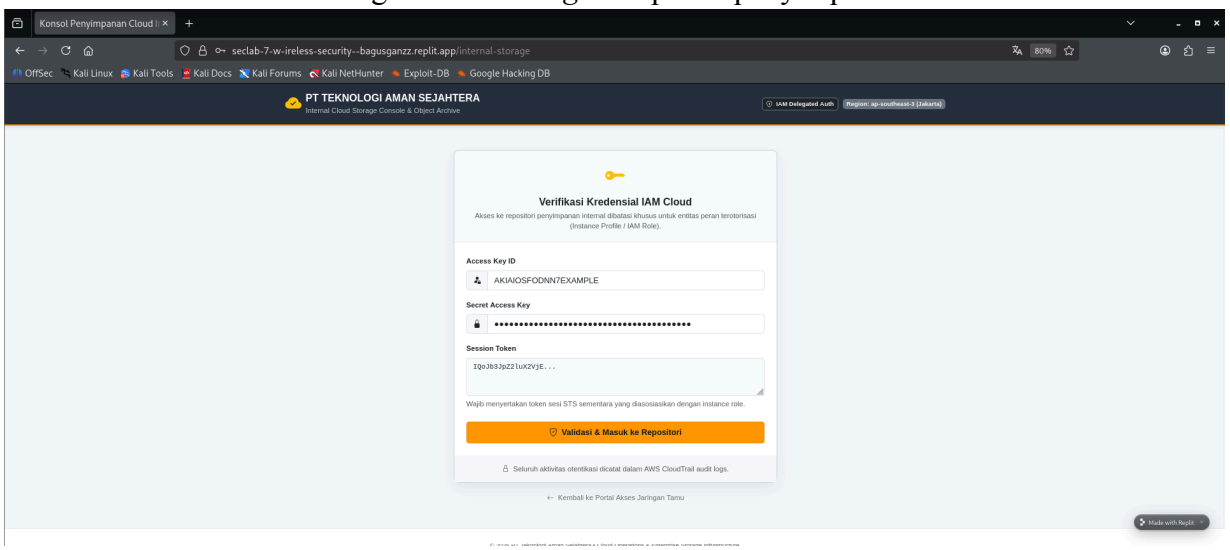


Jawaban :
Inspeksi View Source pada endpoint /internal-storage menunjukkan form dengan method="POST" dan tiga field required: access_key_id (text), secret_access_key (password), dan session_token (textarea), yang nama parameternya merefleksikan skema kredensial IAM sementara AWS (AccessKeyId, SecretAccessKey, SessionToken) hasil

ekstraksi IMDS pada Bagian 1.

Ketiga nilai tersebut diisikan ke field yang bersesuaian dan di-submit via POST ke /internal-storage. Backend diasumsikan memvalidasi kredensial melalui AWS STS (sts:GetCallerIdentity) untuk memastikan token masih valid, sehingga apabila valid, server mengenali request sebagai sesi dari admin-role yang sah dan mengizinkan akses ke penyimpanan internal, meskipun sumber kredensial sebenarnya berasal dari eksploitasi SSRF.

2. **Authentication:** Gunakan kredensial IAM (Access Key, Secret Key, dan Token) yang berhasil Anda curi dari Bagian 1 untuk *login* ke portal penyimpanan internal tersebut.



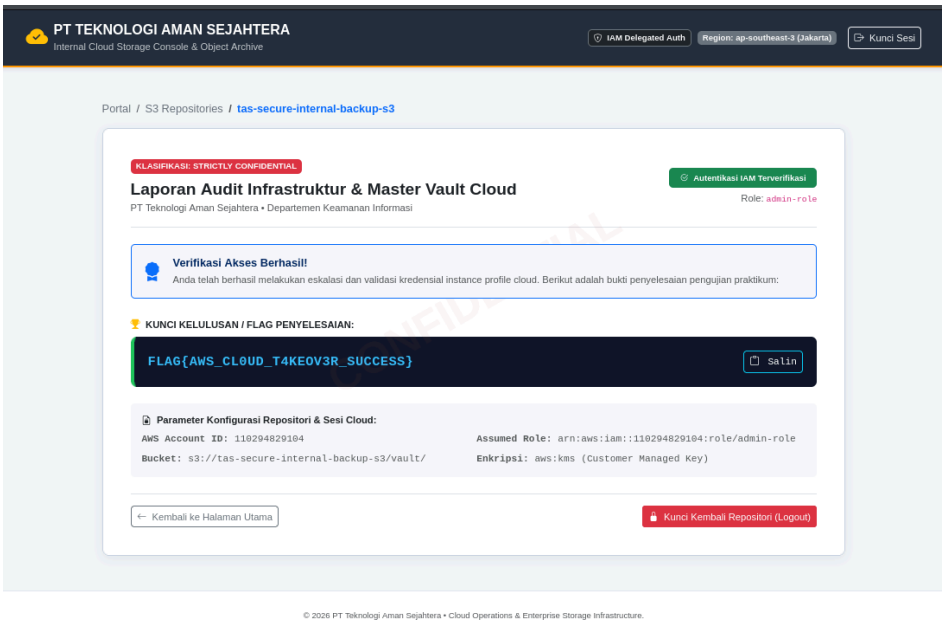
Jawaban :
Autentikasi dilakukan dengan memanfaatkan kredensial IAM sementara (AccessKeyId, SecretAccessKey, Token) yang berhasil diekstrak dari AWS IMDS pada Bagian 1, untuk login ke portal “Konsol Penyimpanan Cloud” pada endpoint /internal-storage.

1. Akses halaman login. Buka URL https://seclab-7-w-ireless-security--bagusganzz.replit.app/internal-storage yang ditemukan pada tahap reconnaissance, menampilkan form “Verifikasi Kredensial IAM Cloud”.
2. Isi field Access Key ID. Masukkan nilai AccessKeyId hasil ekstraksi (contoh: AKIAIOSFODNN7EXAMPLE) ke field access_key_id.
3. Isi field Secret Access Key. Masukkan nilai SecretAccessKey hasil ekstraksi ke field secret_access_key, bertipe password sehingga tampil ter-mask sebagai titik-titik pada form.
4. Isi field Session Token. Masukkan nilai Token (session token sementara) hasil ekstraksi ke field session_token berupa textarea, karena panjang string token STS biasanya terlalu panjang untuk single-line input.
5. Submit form. Klik tombol “Validasi & Masuk ke Repositori” untuk mengirimkan request POST ke endpoint /internal-storage, membawa ketiga kredensial sebagai body/payload request.
6. Validasi oleh server. Backend memvalidasi ketiga kredensial tersebut, kemungkinan melalui pemanggilan AWS STS (sts:GetCallerIdentity atau setara), untuk memastikan

token masih berlaku dan sesuai dengan sesi IAM Role (admin-role) yang terasosiasi.

7. Akses diberikan. Karena kredensial valid, server mengenali request sebagai sesi sah dari entitas terotorisasi dan mengizinkan masuk ke repositori penyimpanan internal, meskipun aktivitas ini tetap tercatat dalam AWS CloudTrail audit logs sebagaimana tertera pada halaman tersebut.

3. **Proof of Capture:** Lampirkan tangkapan layar isi *Secret Document* perusahaan yang berhasil Anda baca.



Jawaban:
Setelah berhasil melewati tahap autentikasi pada langkah sebelumnya, portal mengarahkan ke halaman “S3 Repositories” dengan path breadcrumb Portal / S3 Repositories / tas-secure-internal-backup-s3, menunjukkan bahwa akses telah berhasil masuk ke dalam bucket S3 internal perusahaan. Halaman menampilkan status “Autentikasi IAM Terverifikasi” dengan Role admin-role, mengonfirmasi bahwa sesi yang terbentuk dikenali server sebagai entitas dengan hak akses administratif penuh.

Dokumen yang berhasil diakses berjudul “Laporan Audit Infrastruktur & Master Vault Cloud” dengan klasifikasi “STRICTLY CONFIDENTIAL” milik Departemen Keamanan Informasi, membuktikan bahwa data sensitif perusahaan benar benar dapat dibaca melalui celah ini. Pada bagian “Parameter Konfigurasi Repositori & Sesi Cloud” tercantum detail teknis seperti AWS Account ID 110294829104, bucket s3://tas-secure-internal-backup-s3/vault/, Assumed Role arn:aws:iam::110294829104:role/admin-role, serta enkripsi aws:kms (Customer Managed Key), yang menjadi bukti konkret bahwa eksploitasi berhasil menjangkau resource cloud bertingkat administratif.

Sebagai bukti penyelesaian pengujian praktikum, halaman menampilkan flag FLAG{AWS_CL0UD_T4KEOV3R_SUCCESS} pada bagian “Kunci Kelulusan / Penyelesaian”, yang menjadi proof of capture bahwa rantai eksploitasi mulai dari SSRF, ekstraksi metadata IAM, hingga pengambilalihan akses cloud storage telah berhasil

dibuktikan secara end to end.

BAGIAN 3: REMEDIASI DAN CLOUD ARCHITECTURE - Bobot: 40% Seorang *DevSecOps* harus bisa memberikan rekomendasi infrastruktur yang tangguh.

1. SSRF Patch: Jelaskan secara teknis bagaimana cara menambal celah SSRF pada tingkat kode aplikasi (misalnya: penerapan whitelist URL atau memblokir routing ke IP lokal/privat).

Jawaban :

Kerentanan Server-Side Request Forgery (SSRF) dapat diperbaiki dengan menerapkan validasi URL secara ketat pada kode aplikasi. Salah satu metode yang direkomendasikan adalah URL allowlist, yaitu hanya mengizinkan domain yang telah ditentukan dan dipercaya untuk diakses oleh server. Aplikasi juga harus memvalidasi skema URL, hostname, port, serta alamat IP hasil resolusi DNS. Akses menuju alamat loopback, IP privat, dan alamat link-local seperti 169.254.169.254 yang digunakan AWS Instance Metadata Service harus diblokir apabila tidak diperlukan. Selain itu, aplikasi perlu memvalidasi ulang setiap tujuan pengalihan (redirect) untuk mencegah penyerang mengarahkan koneksi ke alamat internal melalui domain yang awalnya diizinkan. Perlindungan tambahan dapat diterapkan melalui firewall dan aturan egress filtering agar server hanya dapat melakukan koneksi keluar ke layanan yang diperlukan. Dengan kombinasi tersebut, risiko penyalahgunaan fitur diagnostik untuk mengakses jaringan internal dan mengambil informasi sensitif dapat dikurangi secara signifikan.

2. **Cloud Metadata Defense:** Insiden ini terjadi karena pengembang membiarkan layanan IMDSv1 terbuka. Apa perbedaan fundamental antara IMDSv1 dan IMDSv2 pada lingkungan AWS? Jelaskan mengapa mengaktifkan IMDSv2 akan menggagalkan serangan SSRF yang baru saja Anda lakukan.

Jawaban:
AWS Instance Metadata Service memiliki dua versi utama, yaitu IMDSv1 dan IMDSv2. IMDSv1 menggunakan mekanisme permintaan dan respons tanpa mewajibkan token sesi, sehingga aplikasi yang memiliki celah SSRF berpotensi mengakses metadata apabila server dapat menjangkau layanan tersebut. Sementara itu, IMDSv2 menggunakan mekanisme berbasis sesi yang mengharuskan pemanggil memperoleh token melalui permintaan HTTP PUT, kemudian menyertakan token tersebut pada header permintaan metadata berikutnya. Dengan mengaktifkan IMDSv2 dan menetapkan konfigurasi HttpTokens=required, permintaan metadata tanpa token akan ditolak. Mekanisme ini dapat menggagalkan serangan SSRF sederhana yang hanya mampu meneruskan permintaan HTTP biasa tanpa mengirim metode dan header yang diperlukan. Namun, IMDSv2 bukan perlindungan mutlak karena SSRF yang memungkinkan manipulasi metode dan header masih berpotensi menyalahgunakan metadata. Oleh sebab itu, penggunaan IMDSv2 harus disertai validasi URL, pembatasan akses jaringan ke endpoint metadata, serta pemantauan

aktivitas server untuk mencegah kebocoran kredensial IAM.

3. **Least Privilege:** Akses IAM yang Anda curi memiliki identitas `admin-role`. Mengapa memberikan peran (*role*) setingkat administrator ke sebuah mesin *web server* publik adalah praktik yang sangat berbahaya? Jelaskan arsitektur *IAM Role* yang ideal untuk sebuah *web server*.

   Jawaban :
   Memberikan Administrator Role kepada web server yang dapat diakses dari internet merupakan praktik yang sangat berbahaya karena kompromi pada aplikasi dapat berkembang menjadi kompromi terhadap sumber daya cloud lainnya. Apabila penyerang berhasil mengeksploitasi SSRF dan memperoleh kredensial IAM sementara, izin administrator yang terlalu luas dapat memungkinkan akses ke data sensitif, perubahan konfigurasi infrastruktur, penghapusan sumber daya, atau pengelolaan identitas cloud, bergantung pada izin efektif yang dimiliki role tersebut. Untuk mencegah risiko ini, perusahaan harus menerapkan prinsip Least Privilege, yaitu memberikan hak akses minimum sesuai kebutuhan operasional aplikasi. Arsitektur IAM yang ideal menggunakan IAM Role khusus untuk setiap workload, dengan kredensial sementara yang diperoleh melalui EC2 Instance Profile tanpa menyimpan access key permanen di dalam kode aplikasi. Kebijakan IAM harus membatasi tindakan yang diperbolehkan dan resource yang dapat diakses, misalnya hanya mengizinkan aplikasi membaca objek tertentu pada satu bucket S3 apabila memang diperlukan. Selain itu, perusahaan perlu menerapkan trust policy yang ketat, memantau penggunaan izin melalui AWS CloudTrail, dan meninjau kebijakan IAM secara berkala. Dengan pendekatan ini, apabila web server berhasil dikompromikan, dampak serangan dapat dibatasi karena kredensial yang terekspos tidak memiliki kewenangan untuk mengakses atau mengubah seluruh infrastruktur cloud.

BONUS CHALLENGE: SPEED & ELITE HACKER (NILAI TAMBAHAN EKSTRA: +20 POIN) *PERHATIAN: Poin ekstra dari tantangan bonus ini **EKSKLUSIF hanya akan diberikan kepada 3 (TIGA) MAHASISWA PERTAMA** yang berhasil mengumpulkan laporan dengan jawaban yang benar dan lengkap untuk bagian ini. Siapa cepat, dia dapat!*

Tantangan opsional ini dirancang bagi peserta yang mengincar standar *Top Graduate*. Di dunia nyata, peretasan *Cloud* dieksekusi via terminal, dan sistem keamanan sering kali memiliki filter *blacklist*.

1. **The Real-World Cloud Pivot (Terminal Eksploitasi):** Di peretasan nyata, Anda tidak memasukkan kunci Cloud ke formulir web lab. Tuliskan perintah (*command*) spesifik di terminal Linux Anda untuk mengonfigurasi `AccessKeyId`, `SecretAccessKey`, dan `Token` sementara tersebut. Kemudian, sebutkan 1 (satu) perintah *AWS CLI* yang harus dieksekusi untuk memverifikasi identitas otorisasi token Anda (layaknya perintah `whoami` di Linux).

   Jawaban :

```
┌──(kurniawan㉿Kurniawan)-[~]
└─$ printf 'Access Key: %s karakter\n' "AKIAIOSFODNN7EXAMPLE"
printf 'Secret Key: %s karakter\n' "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
printf 'Session Token: %s karakter\n' "IQoJb3JpZ2luX2VjE... "
Access Key: AKIAIOSFODNN7EXAMPLE karakter
Secret Key: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY karakter
Session Token: IQoJb3JpZ2luX2VjE... karakter

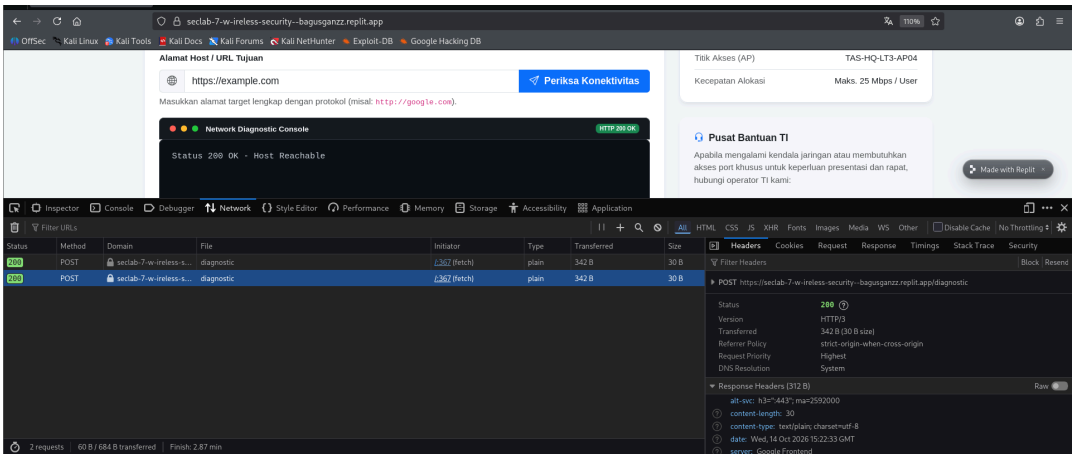
┌──(kurniawan㉿Kurniawan)-[~]
└─$ AWS_ACCESS_KEY_ID="AKIAIOSFODNN7EXAMPLE" \
AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY" \
AWS_SESSION_TOKEN="IQoJb3JpZ2luX2VjE... " \
aws sts get-caller-identity

aws: [ERROR]: An error occurred (InvalidClientTokenId) when calling the GetCallerIdentity operation: The security to
ken included in the request is invalid.
```

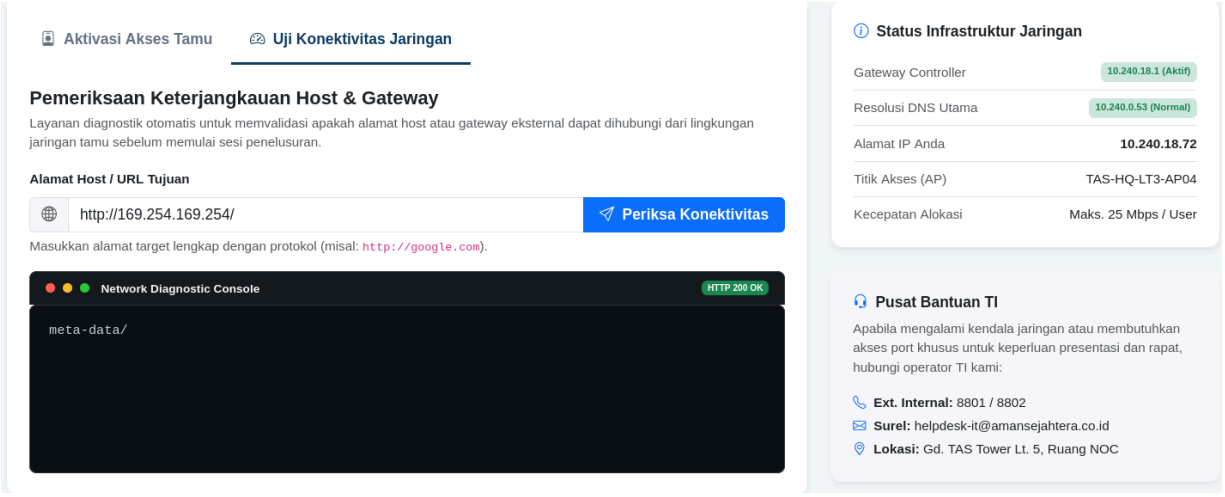
Pengujian aws sts get-caller-identity menghasilkan error InvalidClientTokenId. Setelah diperiksa, nilai access key yang dikembalikan endpoint mengandung penanda EXAMPLE, yang mengindikasikan kemungkinan kredensial placeholder atau simulasi. Keberhasilan respons endpoint tidak membuktikan validitas kredensial terhadap AWS STS.

2. **IP Blacklist Evasion:** Asumsikan pengembang PT Teknologi Aman Sejahtera telah menambal kode *backend* dengan cara memblokir string IP secara harfiah: `if "169.254.169.254" in url: return "Blocked"`. Sebagai penyerang, berikan minimal 3 (tiga) contoh format penulisan *payload* IP yang berbeda untuk mengecoh filter *blacklist* tersebut, namun tetap mengarahkan *request* ke IP Metadata AWS yang sama. Jelaskan mengapa *payload* Anda bisa lolos!

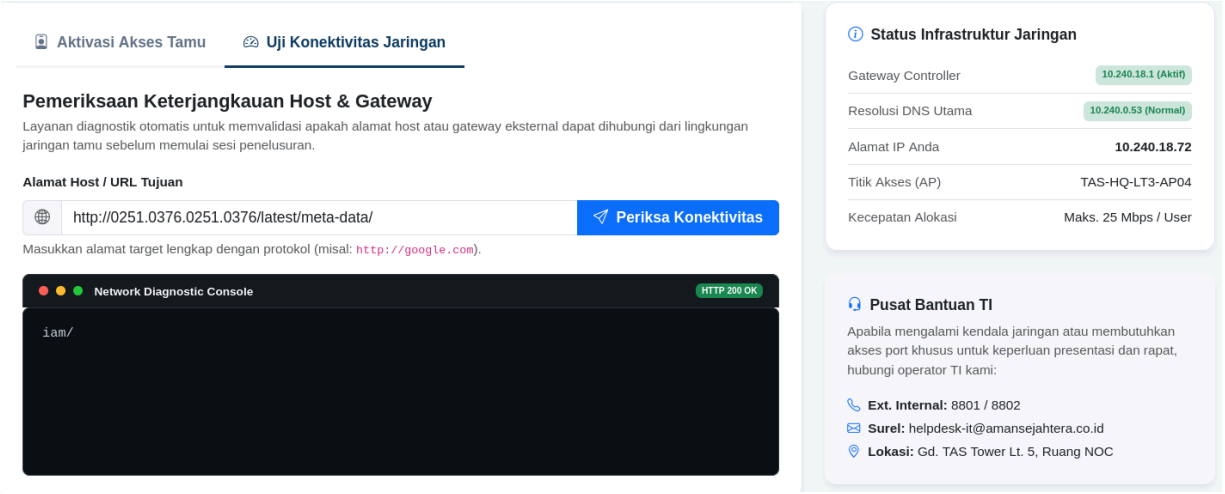
Jawaban :



Pengujian dasar dilakukan melalui fitur Pemeriksaan Keterjangkauan Host & Gateway dengan memasukkan URL https://example.com dan menekan tombol Periksa Konektivitas. Hasil pada Network Diagnostic Console menunjukkan Status 200 OK - Host Reachable, sedangkan tab Network di Developer Tools memperlihatkan permintaan POST ke endpoint /diagnostic dengan status HTTP 200. Hasil ini menunjukkan bahwa aplikasi berhasil memproses permintaan diagnostik dan menampilkan respons kepada pengguna.



Pengujian terhadap alamat http://169.254.169.254/ menghasilkan status HTTP 200 OK dengan respons meta-data/. Hasil ini menunjukkan bahwa endpoint diagnostik mengembalikan respons untuk jalur metadata. Perlu pemeriksaan lebih lanjut untuk memastikan apakah aturan blacklist diterapkan dan apakah respons berasal dari layanan metadata AWS yang sebenarnya.



Pengujian dilakukan melalui fitur Pemeriksaan Keterjangkauan Host & Gateway dengan memasukkan URL http://0251.0376.0251.0376/latest/meta-data/ ke kolom tujuan, kemudian menekan tombol Periksa Konektivitas. Network Diagnostic Console menampilkan respons HTTP 200 OK dengan keluaran iam/, yang menunjukkan bahwa aplikasi mengembalikan respons untuk jalur tersebut. Hasil ini perlu dianalisis lebih lanjut untuk memastikan bagaimana aplikasi menafsirkan format alamat IP dan apakah filter blacklist benar-benar diterapkan.

D. FORMAT LAPORAN DAN PENGUMPULAN Laporan disusun dalam format *Penetration Testing & Remediation Report* bergaya eksekutif yang memuat:

1. **Halaman Sampul:** Nama Kelompok, Daftar Nama Anggota, Nama Divisi, Nama Program.
2. **Executive Summary:** Ringkasan insiden keamanan dan kerentanan infrastruktur

Cloud yang ditemukan.

3. **Proof of Concept (PoC):** Langkah-langkah detail penemuan SSRF, ekstraksi Metadata AWS, eksploitasi Cloud Storage, beserta *screenshot* teknis (Menjawab Bagian 1 & 2).
4. **Remediation & Secure Architecture:** Usulan arsitektur pertahanan Cloud dan perbaikan kode aplikasi (Menjawab Bagian 3).
5. **Format Penamaan Berkas:** Ekspor dokumen ke format PDF dengan nama: `[Nama Kelompok]_[Nama Divisi]_Tugas Modul 7.pdf`.

Total Nilai Maksimal: 100 poin (+20 Poin Ekstra Bonus).